# Wrapper Class

## Module-1




**Dr.M. Sivakumar**

Professor & Technical Trainer / CSE

Saveetha School of Engineering, SIMATS

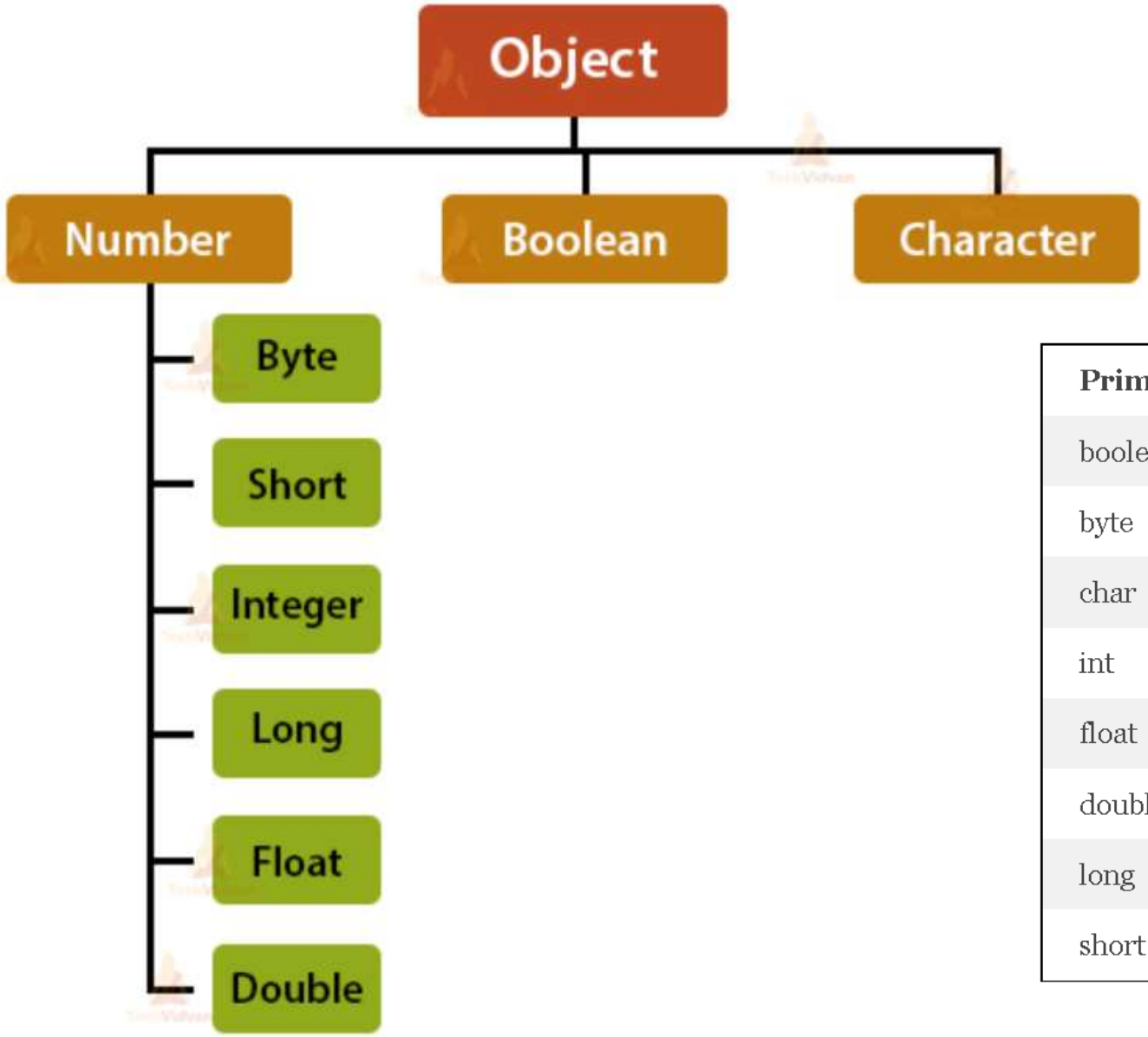
sivakumarm.sse@saveetha.com

+91-9500600868

# Wrapper Class

- As we know, Java is not pure object oriented programming language because of primitive data type.
- Wrapper class in Java is the type of class that provides a mechanism to convert the primitive data types into the objects and vice-versa.
- The process of converting primitive data types into an object is called boxing

# Wrapper Class




## Hierarchy of Wrapper Class

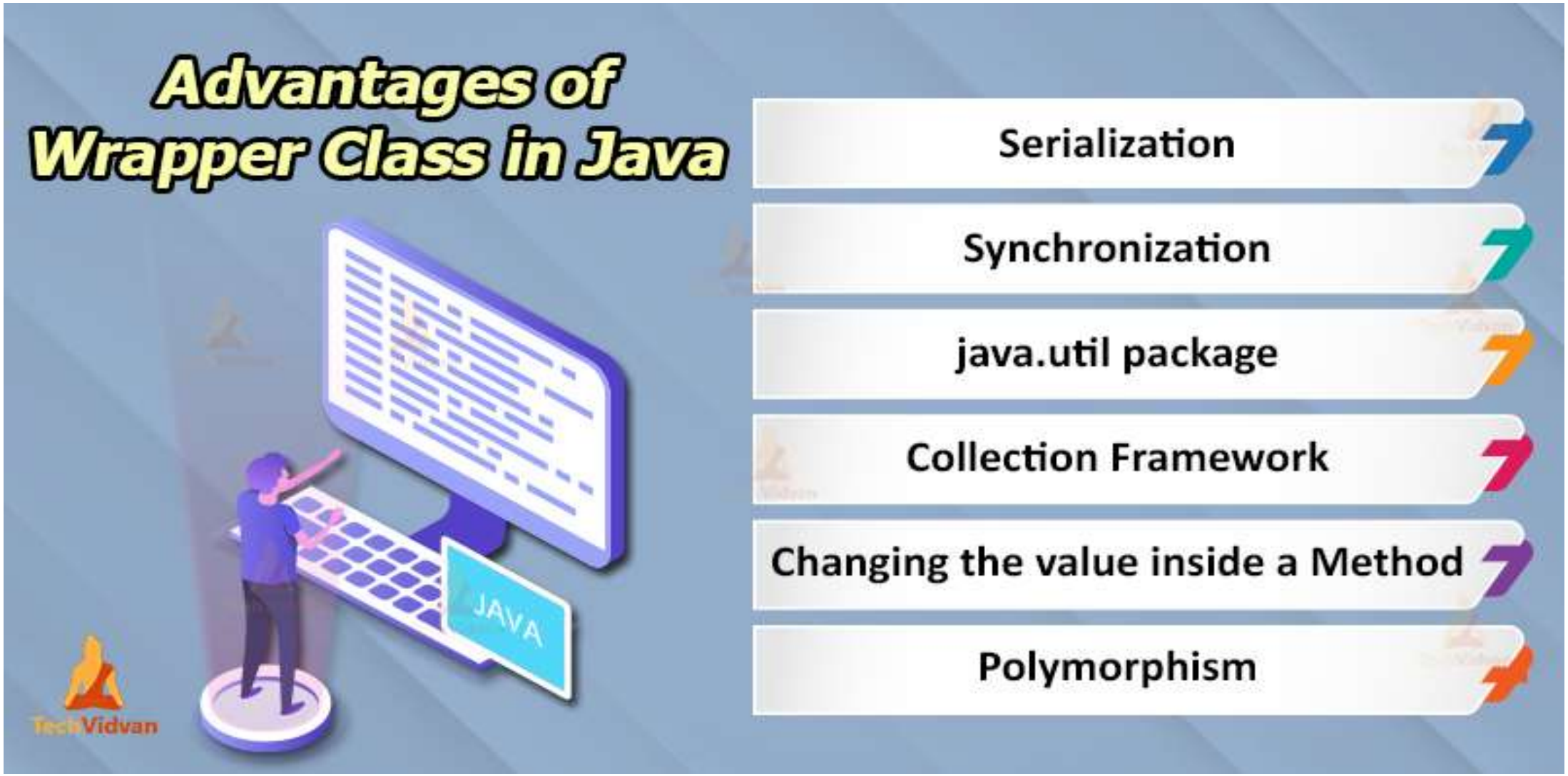
| Primitive Data Type | Wrapper Class | Constructor Argument |
|---|---|---|
| boolean | **Boolean** | boolean or String |
| byte | **Byte** | byte or String |
| char | **Character** | char |
| int | **Integer** | int or String |
| float | **Float** | float, double or String |
| double | **Double** | double or String |
| long | **Long** | long or String |
| short | **Short** | short or String |

# Wrapper Class

## Need of Wrapper Class

- Wrapper classes are used to provide a mechanism to 'wrap' or bind the values of primitive data types into an object.
- Wrapper classes are also used to provide a variety of utility functions for primitives data types like converting primitive types to string objects and vice-versa, converting to various bases like binary, octal or hexadecimal, or comparing various objects.
- We cannot provide null values to Primitive types but wrapper classes can be null. So wrapper classes can be used in such cases we want to assign a null value to primitive data types.

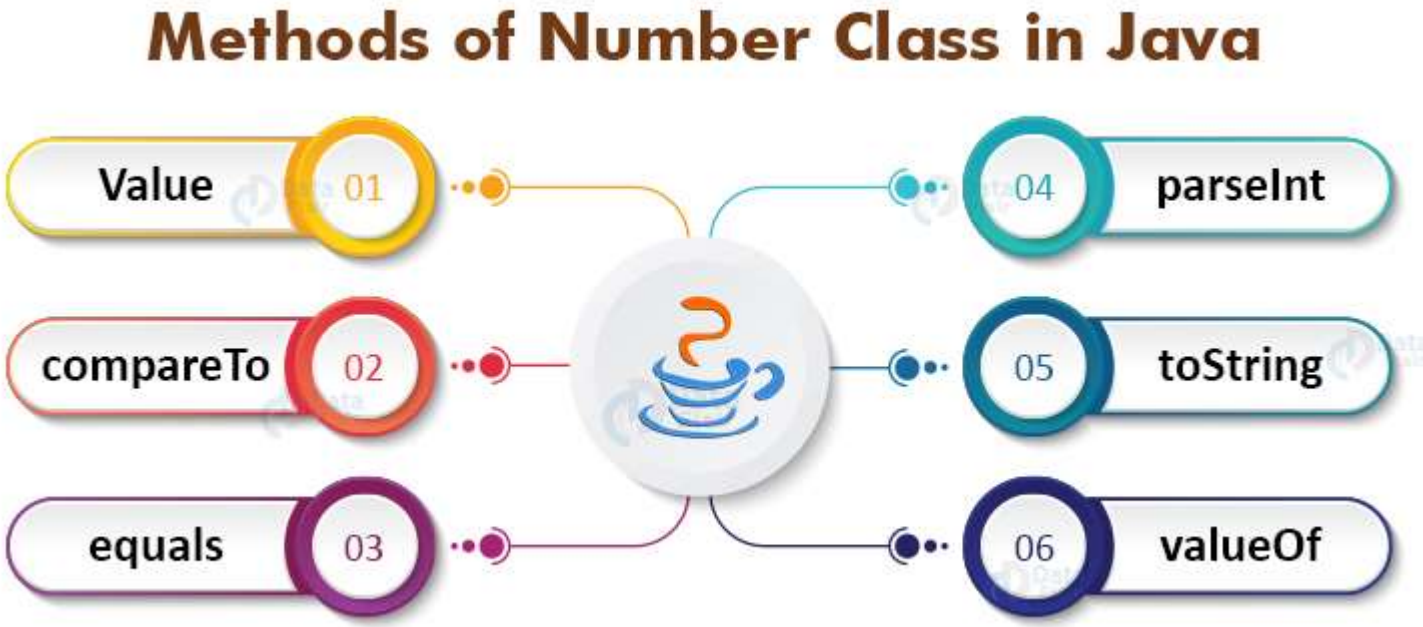
# Wrapper Class



# Wrapper Class

## Number Class

- Java Number class is the super class of all the numeric wrapper classes.
- The Number class contains some methods to provide the common operations for all the sub classes.



# Wrapper Class

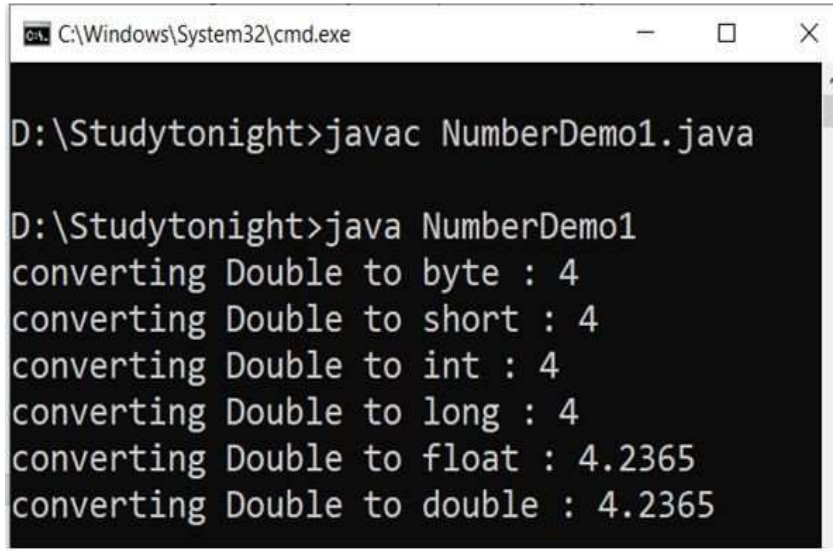
## 1.Value() Method

- This method is used for converting numeric object into a primitive data type.

For example we can convert a Integer object to int or Double object to float type.

| Methods |
| --- |
| byte byteValue() |
| short shortValue() |
| int intValue() |
| long longValue() |
| float floatValue() |
| double doubleValue() |

```
Double d1 = new Double("4.2365");
byte b = d1.byteValue();
short s = d1.shortValue();
int i = d1.intValue();
long l = d1.longValue();
float f = d1.floatValue();
double d = d1.doubleValue();
System.out.println("converting Double to byte : " + b);
System.out.println("converting Double to short : " + s);
System.out.println("converting Double to int : " + i);
System.out.println("converting Double to long : " + l);
System.out.println("converting Double to float : " + f);
System.out.println("converting Double to double : " + d1);
```



# Wrapper Class

## 2.valueOf() Method

- This method is used to get an Integer object representing the specified int value.
- It takes a single int type argument and returns an Integer instance

```
int a = 95;
Integer x = Integer.valueOf(a);
System.out.println("valueOf(a) = " + x);
```

**Output:**
valueOf(a) = 95

## 3. parseInt() Method

- This method is used to parse the specified string argument as a signed decimal integer.
- The characters in the string must all be decimal digits.
- It takes a single argument of string type and returns an int value.

```
String a = "95";
Integer x = Integer.parseInt(a);
System.out.println("parseInt(a) = " + x);
```

**Output:**
parseInt(a) = 95

# Wrapper Class

## 4.toString() Method

- This method returns a String object representing this Integer's value.
- The value is converted to signed decimal representation and returned as a string.
- It overrides toString() method of Object class.
- It does not take any argument but returns a string representation of the value of this object in base 10.

```
int a = 95;
Integer x = new Integer(a);
System.out.println("toString(a) = " + Integer.toString(a));
```

**Output:**

toString(a) = 95

# Wrapper Class

## 5.compareTo() Method

- This method compares the input argument with the Number object.
- It returns 1 (positive number) if the value of Number object is greater than the argument, -1 (negative number) if it is less than the value of argument and 0 if it is equal to the argument.
- But both the argument and the number should be of the same type

```
Integer num1 = new Integer("5");
System.out.println("comparing num1 with 10 is " + num1.compareTo(10));
System.out.println("comparing num1 with 1 is " + num1.compareTo(1));
```

**Output:**

```
comparing num1 with 10 is -1
comparing num1 with 1 is 1
```

# Wrapper Class

## 6.equals() Method

- The equals method checks if the number and the argument passed through the method is not null and equal to the number.
- If both the numbers are equal, it returns true. If not, it returns false.
- They also have to be of the same type.

```
Integer num1 = new Integer("5");
Short s = new Short("5");
Integer num2 = new Integer("5");
System.out.println("Is the short value equal to num1? " + num1.equals(s));
System.out.println("is the integer value equal to num1? " + num1.equals(num2));
```

**Output:**

```
Is the short value equal to num1? false
is the integer value equal to num1? true
```

# Wrapper Class

## Autoboxing and Unboxing in Java

- Java **Autoboxing** is the process of converting a primitive datatype to its specific wrapper class object. This is useful when:
  - Primitive datatype is passed as an argument to a method that expects an object.
  - Variable is assigned to the corresponding wrapper class.
- Java **Unboxing** is the reverse of autoboxing. It is the process of converting the Wrapper class object into its corresponding primitive datatype. This is useful when:
  - Wrapper class object is used where a primitive datatype is expected.
  - It is assigned to a primitive

# Wrapper Class

## Example:

```
System.out.println("Understanding Autoboxing");
                    //This is a primitive datatype
Integer i = 10;
                  //Autoboxing of the primitive value 10 to an object
System.out.println("The integer is " + i);
System.out.println("Understanding Unboxing");
Integer num = new Integer(98);
int unboxnum = num;
             //Unboxing of the num object to a primitive datatype int
System.out.println("The value of the integer is " + unboxnum);
```

**Output:**

```
Understanding Autoboxing
The integer is 10
Understanding Unboxing
The value of the integer is 98
```